Übungsblatt 5

CSS-Layouts, Responsive Design, DOM-Manipulation

5430UE Web and Data Engineering

13. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Beherrschung zentraler CSS-Konzepte wie Selektoren, Spezifität, Box-Modell und Normal Flow
- Entwicklung responsiver Benutzeroberflächen mithilfe von Flexbox, CSS Grid und Media Queries
- Analyse und Umsetzung adaptiver Layoutstrategien für unterschiedliche Endgeräte
- Implementierung interaktiver Webanwendungen mittels DOM-Manipulation und Event-Handling



API Gateway und Fat Clients

API Gateway und Fat Clients

Wir betrachten folgendes Szenario:
Ein E-Commerce-Unternehmen hat seine Anwendung in 8 Microservices aufgeteilt. Die mobile App muss für eine einzige Produktseite Daten von 4 verschiedenen Services abrufen.

1. Erklären Sie kurz, was ein **API Gateway** ist. Welches Problem löst es in diesem Szenario?
2. Erklären Sie kurz, worin sich ein **API Gateway** von einem einfachen **API Proxy (Reverse Proxy)** unterscheidet.
3. Diskutieren Sie **Fat Client vs. Thin Client**: Nennen Sie je zwei Vor- und Nachteile eines Fat Clients für Web-Anwendungen.

CSS Spezifizität

CSS Spezifizität (1/3)

Gegeben sind der folgende HTML-Code und das zugehörige CSS-Stylesheet:

index.html

```
<!DOCTYPE html>
<html lang="de">
   <head>
      <meta charset="UTF-8">
      <title>Spezifizitaet</title>
      <link rel="stylesheet" href="style.css">
    </head>
   <body id="body-main">
      <div id="container">
          <span id="item-1" class="highlight important">Ziel-Text</span>
      </div>
   </body>
</html>
```

CSS Spezifizität (2/3)

style.css

```css
/* 1 */
#body-main div.highlight { color: black; }

/* 2 */
#item-1 { color: blue; }

/* 3 */
span { color: grey; background-color: white; }

/* 4 */
body #container .important { color: cyan; }

/* 5 */
div #item-1 { color: green; }

/* 6 */
.highlight { color: red; }

/* 7 */
```



CSS Spezifizität (3/3)

1. Berechnen Sie für jeden der 8 Selektoren die Spezifität nach dem Schema *(ID, Klasse, Element)*.
2. Ordnen Sie die Selektoren in aufsteigender Priorität (niedrigste zuerst).
3. Welches visuelle Erscheinungsbild ergibt sich für das span-Element bezüglich Textfarbe (`color`) und Hintergrundfarbe (`background-color`)?

Normal Flow und Element-Typen

Normal Flow und Element-Typen

1. Beschreiben Sie das Standardverhalten (Normal Flow) von *Block-Elementen* und *Inline-Elementen* im Browser, wenn keine besonderen Layoutregeln wie Flexbox oder Grid angewendet werden.

2. Gegeben sei der folgende HTML-Code:

```
<p>Ein Absatz mit einem <span>Inline-Text</span>.</p>
```

Was passiert visuell, wenn auf das -Element im CSS die Regel `display: block;` angewendet wird? Ändert sich dadurch die semantische Bedeutung des Elements?

Box-Modell und Selektoren

Box-Modell und Selektoren

Gegeben folgendes CSS:

```css
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 16px;
}
```

1. Wie breit ist `.card` im Standard-Box-Modell (`content-box`)? Begründen Sie.
2. Wie breit wäre `.card` mit `box-sizing: border-box`?
3. Schreiben Sie einen CSS-Selektor, der nur das erste `.card`-Element innerhalb eines `<section>` trifft.

Responsive Layout mit Flexbox

Responsive Layout mit Flexbox

Implementieren Sie ein responsives Layout, das auf Desktop-Monitoren (≥ 768px) drei Spalten nebeneinander anzeigt und auf Mobilgeräten (< 768px) auf eine einspaltige Ansicht umspringt. Nutzen Sie dafür ausschließlich Flexbox-Eigenschaften für den Container `.product-layout` und die Kindelemente `.product-card`.

Ein entsprechendes Projekt finden Sie in Stud.IP unter dem Namen *flex_layout*. Ergänzen Sie dort die fehlenden Deklarationen in der Datei `style.css` an den markierten Stellen.

Hinweis:
Ein sehr gutes Cheat-Sheet zu Flexbox finden Sie unter: https://css-tricks.com/snippets/css/a-guide-to-flexbox/

CSS Grid Layout

CSS Grid Layout (1/3)

1. Erläutern Sie anhand einer Faustregel, wann der Einsatz von CSS Grid und wann der Einsatz von Flexbox für ein Layout sinnvoll ist. Beziehen Sie sich in Ihrer Antwort auch auf das Verhalten der `.product-card`-Elemente aus der vorherigen Flexbox-Aufgabe.
2. Erklären Sie kurz die Funktionsweise der CSS-Funktion `minmax()` in Kombination mit `auto-fit` für ein responsives Grid-Layout. Welcher Vorteil ergibt sich hierdurch im Vergleich zur Verwendung von starren Breakpoints (Media Queries)?

CSS Grid Layout (2/3)

3. **Explizites Raster (mit Media Query):** Gegeben ist der Container `.product-detail`, der ein Bild (`image`), Kurzinfos (`info`) und eine ausführliche Beschreibung (`desc`) enthält. Implementieren Sie das CSS so, dass auf Desktops folgendes 2-spaltiges Raster entsteht:

   - Spalte 1 (`1fr`) enthält oben das Bild.
   - Spalte 2 (`1.2fr`) enthält die Kurzinfos.
   - Die Beschreibung (`desc`) soll in einer zweiten Zeile stehen und **beide Spalten** überspannen.

   Auf mobilen Geräten (`max-width: 700px`) sollen alle drei Elemente einfach untereinander stehen. Nutzen Sie für die Positionierung ausschließlich `grid-template-areas`.

   Ein entsprechendes Projekt mit der Dateistruktur für die Aufgaben c) und d) finden Sie in Stud.IP unter dem Namen *grid_layout*. Ergänzen Sie dort die fehlenden Deklarationen in der Datei `style.css` an den markierten Stellen.

CSS Grid Layout (3/3)

4. **Responsives Raster (ohne Media Query):** Entwickeln Sie nun ein responsives Grid für den Container `.product-grid`, das seine Spaltenanzahl automatisch an den verfügbaren Platz anpasst, *ohne* Media Queries zu verwenden. Die Spalten sollen eine Mindestbreite von `250px` haben und sich ansonsten gleichmäßig ausdehnen. Stellen Sie durch eine weitere Grid-Eigenschaft sicher, dass alle automatisch erzeugten Zeilen mindestens `150px` hoch sind, sich bei viel Inhalt aber automatisch vergrößern.

DOM-Manipulation und Events

DOM-Manipulation und Events

Implementieren Sie folgende Funktionalität in JavaScript:

Ein Button (`id: toggle-theme`) soll beim Klick die Klasse `dark-mode` auf dem `<body>` togglen (siehe `toggle`-Funktion), d.h. zur Liste der Klassen des `<body>`-Elements hinzufügen, wenn die Klasse nicht vorhanden ist und andernfalls entfernen. Zusätzlich soll der Button-Text zwischen „Dark Mode" und „Light Mode" wechseln.

Ein vorbereitetes Projekt finden Sie in Stud.IP unter dem Namen *moduswechsel*. Ergänzen Sie eine JavaScript-Datei `script.js` mit entsprechendem Code. Nutzen Sie einen EventListener, um auf Klicks des Buttons mit der ID `toggle-theme` zu reagieren.

Hinweis: Die Dateien `index.html` und `style.css` sind bereits vollständig vorgegeben und müssen nicht verändert werden.

Materialien

Materialien zum Übungsblatt

- Aufgabe *Responsive Layout mit Flexbox*: flex_layout.zip
- Aufgabe *CSS Grid Layout*: grid_layout.zip
- Aufgabe *DOM-Manipulation und Events*: moduswechsel.zip